

Phase 0 — Engineering Foundation & Project Setup

Objective

This phase marks the transition from planning to implementation.

No business features will be developed.

Instead, the objective is to build an engineering environment that resembles how a production backend project begins.

Unlike tutorials that immediately start writing APIs, this project will first establish a professional development workflow.

By the end of Phase 0, the project should have:

- Development environment
- GitHub repository
- Git workflow
- CI pipeline
- Project management
- Code quality tools
- Containerized infrastructure
- Development tooling
- Documentation
- Local production-like environment

Every subsequent feature will be built on top of this foundation.

Learning Objectives

After completing Phase 0, you should understand:

- How professional backend repositories are structured.
- How GitHub is used beyond source control.
- Why CI/CD exists.
- How Docker fits into development.
- Why linting and formatting matter.
- How environment configuration works.
- How production projects maintain code quality.
- How infrastructure is managed locally.

The emphasis is on understanding **why** each tool exists before learning **how** to configure it.

Engineering Philosophy

Throughout Phase 0, every decision should satisfy the following principles.

Learn Before Implementing

Every tool introduced should answer three questions.

1. What problem does it solve?
2. Why do production systems use it?
3. Why was it selected over alternatives?

Configuration without understanding has little educational value.

Production Parity

The local environment should resemble production as closely as possible.

Examples:

Development

Docker Compose

↓

Node.js

↓

Redis

↓

Supabase PostgreSQL

Production

Kubernetes

↓



Only the infrastructure changes.

The application code remains identical.

Free-Tier First

The project is designed around a zero-cost development strategy.

Whenever possible, infrastructure will be hosted locally.

Component	Strategy
PostgreSQL	Supabase Free Tier
Redis	Local Docker
BullMQ	Local Docker
Prometheus	Local Docker
Grafana	Local Docker
GitHub Actions	Free Minutes
GitHub Container Registry	Free
Docker	Local
Node.js	Local

No architectural decision should require paid infrastructure during development.

Phase 0 Roadmap

Phase 0 is divided into multiple implementation milestones.

Phase 0.1

↓

Developer Workstation

↓

Phase 0.2

Git & GitHub

↓

Phase 0.3

Project Management

↓

Phase 0.4

Repository Standards

↓

Phase 0.5

Node.js Project

↓

Phase 0.6

Development Tooling

↓

Phase 0.7

Docker Infrastructure

↓

Phase 0.8

Configuration Management

↓

Phase 0.9

CI Pipeline

↓

Phase 0.10

Project Verification

Each milestone builds upon the previous one.

Nothing will be skipped.

What Will Be Built

By the end of Phase 0, the repository should already resemble a professional production backend.

```
seat-sync/  
├─ backend/  
├─ frontend/  
├─ docs/  
├─ infrastructure/  
├─ scripts/  
├─ .github/  
│   ├─ workflows/  
│   └─ ISSUE_TEMPLATE/  
│       └─ PULL_REQUEST_TEMPLATE.md  
├─ docker-compose.yml  
├─ README.md  
├─ LICENSE  
├─ .gitignore  
├─ .editorconfig  
├─ .env.example  
├─ CONTRIBUTING.md  
├─ SECURITY.md  
└─ CHANGELOG.md
```

Although very little application code exists, the engineering foundation should already be complete.

Practical Learning Approach

Every Phase 0 milestone will follow the same structure.

Part 1

Concept

Learn the engineering concept.

Examples:

- What is Docker?
 - Why GitHub Actions?
 - What is ESLint?
 - Why Husky?
 - Why Conventional Commits?
-

Part 2

Industry Perspective

How production teams use it.

Typical mistakes.

Trade-offs.

Alternatives.

Part 3

Implementation

Hands-on setup.

Installation.

Configuration.

Verification.

Part 4

Deep Dive

What actually happens internally.

How the tool works.

Common interview questions.

Common production issues.

Best practices.

Part 5

Project Integration

Connect the tool to the Seat Sync project.

Every configuration should have a clear purpose.

Expected Deliverables

At the end of Phase 0, the project will have:

- ✓ Professional Git repository
- ✓ Repository standards
- ✓ GitHub Project
- ✓ Labels
- ✓ Milestones
- ✓ Issue Templates
- ✓ Pull Request Templates
- ✓ Branch Protection

- ✓ Semantic Versioning
- ✓ Conventional Commits
- ✓ Node.js project
- ✓ TypeScript
- ✓ ESLint
- ✓ Prettier
- ✓ Husky
- ✓ lint-staged
- ✓ Docker Compose
- ✓ Redis
- ✓ Supabase
- ✓ Environment configuration
- ✓ GitHub Actions
- ✓ Initial CI pipeline
- ✓ Development documentation

No business feature will exist yet, but the engineering quality of the repository should already match that of a production-ready project.

Phase 0 Milestones

Phase	Topic
Phase 0.1	Developer Workstation & Required Software
Phase 0.2	Git & GitHub Repository Setup
Phase 0.3	GitHub Projects, Issues & Project Management
Phase 0.4	Repository Standards & Documentation

Phase	Topic
Phase 0.5	Initializing the Backend Project
Phase 0.6	Development Tooling (TypeScript, ESLint, Prettier, Husky, lint-staged)
Phase 0.7	Docker & Local Infrastructure
Phase 0.8	Configuration & Environment Management
Phase 0.9	GitHub Actions & Continuous Integration
Phase 0.10	Foundation Verification & Architecture Audit

Practical Expectations

This project is not intended to produce the fastest implementation.

Instead, every milestone should leave you capable of explaining:

- Why the tool exists.
- What problem it solves.
- How it works internally.
- Why it was selected.
- What alternatives exist.
- How it is used in production.
- How to configure it confidently from scratch.

By the end of Phase 0, you should be able to create the engineering foundation of a backend project without relying on tutorials.

Deliverables

At the completion of this phase, the implementation roadmap is fully established.

Beginning with **Phase 0.1**, every tool, configuration, and engineering practice will be introduced incrementally, with practical implementation always accompanied by the underlying concepts, production use cases, trade-offs, and best practices.

Only after this engineering foundation is complete will the project move to **Phase 1 — Building the Backend Foundation**, where the first line of application code will be written.